Full Stack Vibe Coder Intern Assessment

ASSESSMENT INSTRUCTIONS

Position: Full Stack Vibe Coder Intern

- **Start Time: 1st April 2026, 10:00 AM (IST)**
- **Deadline: 4th April 2026, 10:00 AM (IST)**
- **DEADLINE - NO EXTENSIONS GRANTED**

Submission Requirements:

- **File Name:** Full Stack Vibe Coder Intern_[Your_Name] _March2026
- **Email to:** [career@purplemerit.com] with subject Full Stack Vibe coder Intern Assessment Submission - [Your Name]"

❖ **LATE SUBMISSIONS ARE DISQUALIFIED.**

Assessment Overview

What We're Evaluating

We're looking for strong ability across:

1. **Web design & "vibe"** (typography, layout, spacing, theming, UI states)
2. **Responsiveness** (must be excellent on **mobile + tablet**, not just desktop)
3. **Full-stack execution** (auth, CRUD, persistence, API correctness, production deploy)
4. **Product thinking** (sensible defaults, UX flow, edge cases, performance)

Tech Constraints (Required)

Deployment

- Must be deployed on **Netlify**
- Backend must be implemented via **Netlify Functions** (serverless API)

Database

- You may use **any database you prefer** (MongoDB / PostgreSQL / MySQL / Supabase / etc.)
- **Extra points if you use PostgreSQL** (clean schema + constraints + migrations)
- Data must be persisted in the database (no "JSON file only" persistence)

Security & Secrets

- No secrets committed to the repo
- Use Netlify environment variables for DB connection strings and secrets

Project Brief: VibeKit Studio

Tagline: "Generate a theme, build a mini-site, publish it."

Build a web app where a user can:

1. select/generate a **design theme ("vibe")**
2. apply it to a **mini-site** (simple page builder)
3. **publish** it to a public URL that looks polished and loads fast

Required App Sections (MVP)

1) Public Website

A. Landing Page — /

Must include:

- A polished marketing-style landing page for **VibeKit Studio**
- Clear CTA: **Create your first page**
- Showcase **3 example themes/templates** (can be static)

B. Published Page — /p/:slug

- Public view of a user's **published mini-site**
- Must match the selected theme exactly (preview = published)
- Must be **fast and responsive**
- Track **view count** (persisted in DB)

2) Authentication + Dashboard

A. Auth

- Email + password signup/login
- Passwords must be hashed (bcrypt or equivalent)
- Sessions: JWT (httpOnly cookie preferred) or token-based (explain in README)

B. Dashboard — /app

Must support:

- Create a new Page
- List pages (show status: draft / published)
- Duplicate (clone) a page

3) Page Builder (Core Functionality)

A **Page** is a themed mini site with a fixed structure:

Page sections (minimum 4)

1. **Hero**: title, subtitle, primary button (text + URL)
2. **Features**: 3–6 feature cards (title + short description)
3. **Gallery**: 3–8 images (image URLs are fine)
4. **Contact**: form (name, email, message)

Editor requirements

- **Live preview**
- Preview toggle: **Desktop / Tablet / Mobile**
    - Must actually change layout width (not just zoom)
- Reorder sections (up/down buttons are OK)
- Auto-save OR explicit Save with a clear "Saved" state
- Publish / Unpublish toggle
- Unique slug (auto-suggest from title; handle collisions gracefully)

4) Theme System ("Vibe" Generator) — Design-Heavy Requirement

Provide at least **6 vibe presets**, for example:

- Minimal / Editorial
- Neo-brutal
- Dark / Neon
- Pastel / Soft
- Luxury / Serif
- Retro / Pixel

Each preset must define:

- Color palette: background / surface / text / accent

- Typography: font pairing OR a clearly distinct type scale/weights
- Spacing + radius style
- Button style (solid/outline/glow/etc.)

Implementation requirement

- Use **CSS variables (design tokens)** and apply them consistently across components
- Published page must render **identically** to preview (same tokens, same layout rules)

Choose any 3 "design extras"

- Micro-interactions (hover/focus/pressed states)
- Subtle animations (section reveal, button interaction)
- Dark mode (global or per-theme)
- Skeleton loaders
- Accessibility pass (focus rings + contrast)

Mobile + Tablet Responsiveness (Non-Negotiable)

We will evaluate responsiveness heavily.

Viewports we will test

Your UI must look intentional and function correctly at:

- **Mobile:** 320px–480px
- **Tablet:** 768px–1024px
- **Desktop:** 1280px+

Must-pass responsiveness criteria

- No horizontal scrolling at **320px** and **768px**
- Touch targets for primary actions are ~**44px** or larger
- Navigation works without hover (no hover-only interactions)
- Modals/menus/forms are usable and scroll-safe on mobile/tablet
- Typography and spacing scale appropriately (not tiny text on tablet)
- Layout changes meaningfully (e.g., 1 col mobile, 2 col tablet, 3 col desktop where appropriate)

Required submission proof (optional but adds extra point)

Provide a **1–2-minute screen recording** demonstrating:

- Landing page at mobile + tablet widths
- Editor preview toggle switching to mobile + tablet
- Published page /p/:slug on mobile + tablet
- One end-to-end flow: create → preview → publish → open public URL

Backend/API Requirements (Netlify Functions)

Implement a production-style API layer.

Minimum entities to persist

(Names are flexible; structure matters.)

- users
- pages
- pageViews (or viewCount on pages)
- contactSubmissions (if storing contact form messages)

Required endpoints (example)

Auth:

- POST /api/auth/signup
- POST /api/auth/login
- POST /api/auth/logout

Pages (authenticated):

- GET /api/pages
- POST /api/pages
- GET /api/pages/:id
- PUT /api/pages/:id
- POST /api/pages/:id/publish
- POST /api/pages/:id/unpublish
- POST /api/pages/:id/duplicate (or equivalent implementation)

Public:

- GET /api/public/pages/:slug
- POST /api/public/pages/:slug/view (increments views in DB)

Contact:

- POST /api/public/pages/:slug/contact
    - Must work end-to-end:
        - Store submission in DB, and/or send via email provider
        - If you send email, still log to DB

Security expectations

- Users can only read/write their own pages
- Publish/unpublish enforced server-side (don't trust the client)
- Validate inputs server-side
- No DB credentials in client-side code

Deployment Requirements (Netlify)

Your submission must be fully functional in production:

- Deployed URL works (no broken routes)
- Refreshing deep links works (e.g., /p/my-page)
- Netlify Functions work in production
- Env vars configured (examples):
    - DATABASE_URL or MONGODB_URI
    - JWT_SECRET
    - Any provider keys (optional)

Deliverables (What to Submit)

1. **Netlify Live URL**
2. **GitHub Repository** (public or private with access)
3. **README.md** including:

- Local setup instructions
- Env vars required
- Deployed URL
- Test user credentials (or allow signup)
- Brief "Tradeoffs + what I'd improve next" (5 bullets)

4. **Mobile + Tablet responsiveness screen recording** (1–2 minutes)

- Clean, readable UI with consistent spacing and type scale
- Thoughtful empty/loading/error states
- Mobile and tablet feel first-class
- Secure handling of auth and secrets
- A production-like deployment anyone can open and use

*Good luck we're excited to see how you approach this as a **vibe coder**. There's no single "right" answer. We're evaluating your ability to combine **design taste and full-stack execution**: thoughtful UI decisions, clean responsive implementation (especially on **mobile + tablet**), solid working functionality end-to-end, and your ability to take an ambiguous product brief and **ship a polished, production-ready experience**.*